School of Computer Science
University of Nottingham

COMP2001 – Artificial Intelligence Methods
Spring 2026

Dr Warren G Jackson and Prof Ender Özcan

Information

Component weight:	30% of the module (60% of the coursework)
Project Deliverable (Part 1 of 2)	
Expected workload:	45 hours
Release date:	16th of March 2026
Submission date:	28th of April 2026 (3pm)
Marks / Feedback release date:	N/A – see associated in-lab practical exam
Submission method:	Moodle submission – an archive (ZIP) of your problem implementation and selection hyper-heuristic to include all code, libraries, instances, and any other files that are required to run your code on a university lab machine.
Late submission policy:	Late submission of the deliverable will incur the standard late penalty and will be applied to the grade of the in-lab practical exam (see below). Late submissions of the deliverable after 3pm on the day of the in-lab exam is not accepted and a mark of 0 will be awarded for TTA2 for any submission after 08/05/2026 at 3pm. Late submission of exam scripts is not accepted (in-lab exam), and a mark of zero for TTA2 will be awarded for an exam script submitted after the deadline.
In-lab Practical Exam (Part 2 of 2)	
Expected workload:	5 hours
"Release date":	8th of May 2026
Submission date:	8th of May 2026 before the end of the in-lab practical exam session.
Marks / Feedback release date:	8th of June 2026
Submission method:	Moodle Quiz
Late submission policy:	Late submission of exam scripts is not accepted (in-lab exam), and a mark of zero for TTA2 will be awarded for an exam script submitted after the deadline. Note here that exam scripts refer to the Moodle quiz submission.

Overview

In this coursework project, you will implement a HyFlex compatible implementation of a specified optimisation problem as well as design and implementation of a learning-based selection hyper-heuristic (the deliverable – part 1 of 2). You will then use your own implementation during the associated in-lab practical exam (part 2 of 2) to solve an unseen instance of the problem (further details below) where you will be given the opportunity to demonstrate your design and understanding of the project via a Moodle quiz with a combination of open textbox questions, multiple choice questions, and dropdown questions.

Requirements

See appendix.

Getting Started

To get started with this coursework, please read this document in full and then **see the Moodle page which contains the template code which you will need to use as a starting point** to complete the implementation of the Open-top Bus Routing problem domain compatible with the HyFlex framework. You will find that the `OBRDomain` class implements other interfaces as well and this is to ensure that your implementation prints out certain outputs which you will be required to provide during the in-lab practical exam in May.

You have the option to complete the practical test either on your own device or using a lab computer. To ensure your implementation is compatible with the lab computers it is essential that you use Java version 21 (as detailed in the computing sessions) to avoid any problems with running your code during the practical exam.

It is highly recommended that you use the School's Gitlab server to keep your work backed up but please do not make your work publicly available, e.g. on github.com, as you risk opening yourself up to academic misconduct.

The final lab exercises were designed to familiarise yourself with the HyFlex framework, if you have not done those yet it is recommended you do those before making a start on this coursework.

Recommendations

Start with implementing the Open-top Bus Routing Problem before moving on to the design and implementation of the learning-based selection hyper-heuristic. Any performance improvements made in the domain layer could influence the optimal configuration of your hyper-heuristic.

After you have implemented the Open-top Bus Routing Problem, you should **design and implement your own learning-based selection hyper-**

heuristic that is different from those introduced in the labs or those made publicly available.

During the practical exam you will be asked to run your hyper-heuristic on an **unseen** instance of the OBR problem (provided during the practical test) for a **single** trial for a duration of **5 nominal minutes** where the initial solution is generated randomly (as opposed to constructively) hence you should design and calibrate your algorithm accordingly. At a minimum this would entail performing some parameter tuning on the provided instances for the appropriate experimental conditions.

If you have any questions, ask in the Moodle forum so that we can address these to all students. There will be a Q&A for any frequently asked questions on the Moodle page.

Submission Instructions

Part 1 of 2 (Deliverable)

Once you have completed the implementation of the **Open-top Bus Routing Problem** and your **learning-based selection hyper-heuristic** and you are happy with your (parameter/algorithm) configuration, you should submit your code to Moodle. Ideally this would be a ZIP archive of your entire project package so you can download and run it with minimal setup overhead during the practical exam. It is a requirement that you do this submission before you will be allowed access to the exam itself.

Note that the code that you use during the in-lab practical exam must be the one submitted to Moodle as above. We reserve the right to check your code and any such discrepancies in your answers that are found when running your submitted code will be overridden.

Part 2 of 2 (In-lab practical exam)

You will answer a series of questions, some of which will require you to run your code as submitted in part 1, which will be made available via an in-person in-lab Moodle quiz on the 8th of June – exact rooms and times to appear on Moodle closer to the date during the timetabled lab hours.

Assessment Criteria

This project is assessed based on your responses to the in-lab practical exam where you will be using your implementation of the problem domain and learning-based selection hyper-heuristic.

50% of the project mark will come from how effectively your learning-based selection hyper-heuristic solves the unseen problem instance hence a correct and efficient implementation of the domain alongside a well-designed learning-based selection hyper-heuristic is essential.

A project that can produce a solution to the unseen problem instance that improves upon the **initially generated solution** (using RANDOM initialisation, not the constructive method) will attract at least a passing mark (>= 20% out of 50%) for this component.
Additional marks will be awarded proportionally based on the level of improvement demonstrated, with solutions categorised as: • OK: solutions in the lower half of all submitted results • Good: solutions performing better than the bottom 50%, up to the top 10% • Best: solutions within the top 10%

If we need to re-run your project on the unseen problem instance due to implementation issues (such as an incorrect objective function) or if you solved a problem instance other than the provided unseen problem instance, then the original mark from this component will be reduced by 20%.

The remaining 50% of the project mark will come from your answers to questions that assess your understanding of the project, correctness of a subset of project components, and quality of your design choices. The details of these questions are reserved until the in-lab practical exam itself as we are assessing your ability to independently apply the concepts, methods, and best practices taught throughout the course to design and implement an effective AI method for solving the given problem. We want to emphasise that during the practical exam you will need to:

- Perform a few **simple** modifications to your implementation. For example, you may be asked to modify the "Number of internal iterations" setting for each of the "Depth of search range" for all local search (hill climbing) type heuristics.
- Run a piece of code that we will be providing you, which will call selected algorithmic component(s) such as the crossover type heuristic which you are expected to implement.

It is therefore essential that your domain implementation conforms to the HyFlex ProblemDomain API, i.e. the "ProblemDomain" interface, and the "InLabPracticalExamInterface", and your selection hyper-heuristic to the HyperHeuristic API "HyperHeuristic". Any implementation must adhere to the interfaces provided in the template code provided on Moodle. You **must not** modify any of the included interfaces.

Learning outcomes

Transferrable Skills:

- The ability to use AI techniques to solve problems.

Professional Skills:

- Enhanced skills to write software to implement an AI method, the ability to evaluate available AI tools and techniques and select those appropriate to a given situation.

Transferable Skills:

- Problem solving, ability to compare and contrast based on understanding and experience of AI programming tools and techniques.

Getting help

Help will be provided in the way of:

- In-person computing sessions 20th March and 27th March, and
- Moodle forums asynchronously.
- Please do not email queries to the module convenors – you will be asked to submit your queries to the Moodle forum.

An additional in-person session will be held on 1st May during the timetabled computing session for:

- Support for getting projects working on lab computers.
- Late submissions.

Academic Integrity

This is an individual coursework. You must not work on your implementation within a group or with AI tools, but you may use AI and group discussions to assist with your domain / hyper-heuristic design(s). (See AI Statement).

AI Statement

This statement gives explicit written consent that the use of Microsoft's Copilot AI tool (link here), which is available through your University login, **is** permitted for:

- Assisting with the implementation of the problem domain
- Assisting with the design and implementation of your learning-based selection hyper-heuristic

However, you **are not** permitted to use AI tools to write/vibe/generate/autocomplete the code itself. Other AI tools - including but not limited to ChatGPT, Claude, DeepSeek, Doubao, Gemini, Grok, IntelliJ AI assistant - are not permitted in this coursework.

During the in-lab exam, AI tools are strictly forbidden and must be closed/disabled/uninstalled prior to starting the exam. (For example, see the following to disable AI assistant in IntelliJ: https://www.jetbrains.com/help/idea/disable-ai-assistant.html).

It is advisable to implement the project on your own as you will need to understand how each component works in case you are asked to explain or modify some part of your code during the practical exam. An over-reliance on such AI tools could mean that you may not fully understand how each component of your implementation works and hence be unable to explain when required to do so during the practical exam.

Appendix – COMP2001 Project 2026

Implementation of a HyFlex compatible Open-top Bus Routing Problem Domain and learning-based selection hyper-heuristic method.

You must submit your implementation and selection hyper-heuristic to Moodle before the deliverable deadline and use this exact implementation during the in-lab practical exam. Submitted code may be crosschecked with your responses so it is essential that the code submitted is the one used during the in-lab practical exam.

A1 – Description of the problem

The *Open-top Bus Routing problem* (OBR) is an NP-hard combinatorial optimisation problem which concerns a single Open-top tourist bus that needs to visit all tourist points-of-interest using the minimum distance to increase customer satisfaction and minimise operating costs.

Problem description: Given a set of n points-of-interest, A, a relation, L, which maps all points-of-interest to physical locations by x and y coordinates, and a main bus depot, D, with location, (D_x, D_y) where the bus begins and ends its journey, the problem is to find an ordering of all points-of-interest which minimises the total distance the bus needs to travel, starting and ending at the depot and visiting all points-of-interest.

The distance between any two points-of-interest is given by the following formula – **note the ceiling operator**:

$$L(a_i, a_j) = \left\lceil \sqrt{(a_i[x] - a_j[x])^2 + (a_i[y] - a_j[y])^2} \right\rceil$$

A2 – Task and Requirements

The aim of this project is to:

1. Implement a fully working HyFlex compatible version of the OBR problem domain, and
2. Design and implement a learning-based selection hyper-heuristic which can be used to solve any given instance of the OBR problem.
    - The selection hyper-heuristic must use a learning mechanism for selecting the low-level heuristics. Submission of a selection hyper-heuristic that does not use any learning will result in a zero mark for this component.

Note that your implementation must NOT be multithreaded as this will circumvent the computational time budget and give you an **unfair advantage** over other students. Any code that contains multithreaded code will be re-ran post the practical in-lab exam locked to a single core and **incur the 20% penalty** as described in the assessment criteria. Examples of such code includes but is not limited to:

- Thread
- Runnable
- Executor
- Stream.map()
    - If you want to use the stream API make sure your stream is sequential using Stream.sequential().map()

A2.1 – HyFlex Compatible OBR Implementation

You should implement a full suite of components to read, encode, modify, and solve instances of the OBR problem. You should refer to the HyFlex API to ensure that your implementation is HyFlex compatible. As a starting point, a zip archive of some skeleton code is provided on Moodle. Your implementation should have at a minimum the features and components explained below.

Which version of HyFlex should I use?

There is an issue with the Java timer used in the original HyFlex and certain modern computing devices/operating systems which causes timer drift. This matters because your hyper-heuristic will run for some non-deterministic amount of time longer than the time limit which means your results are invalid, and when asked during a timed exam to run your hyper-heuristic, this could take longer than the exam itself meaning you get no result to answer that/those question(s). There is a **TimerTest.jar** file on Moodle which you should run from the command line via **java -jar TimerTest.jar** which will tell you if the timer significantly deviates from real time. To run this test, restart your computer, wait for everything to load, and close any programs/apps that may automatically start. Then, run the Java jar file via the command line and wait at least 30 seconds. We believe that the problem is limited to Windows 11 laptops when ran on battery power or when the power plan is set to an equivalent of power saver,

but this may evolve to other configurations/operating systems/devices over time.

If the timer does not deviate significantly, you should use the **HyFlex_1.0.0_ThreadTimer.jar** which is the original CHeSC 2011/HyFlex library implementation, otherwise you should use the **HyFlex_1.0.1_WallTimer.jar** library which only uses the wall clock time (ensure you close as many background processes as possible when running timed experiments to maximise your computational power during the time limit).

Problem Domain

The OBRDomain class contains skeleton code for a subclass of the HyFlex ProblemDomain class implementing all the problem domain specific operations in accordance with the HyFlex API specification. You must implement all methods in this class to complete the required features of the problem domain base class. There are also methods from the Visualisable interface which will allow you to visualise your solutions by running the provided SR_IE_VisualRunner class, and methods from the InLabPracticalExamInterface which you need to implement so you can print out various details that may be asked during the practical exam.

loadInstance(int instanceId) Method

The `loadInstance(int instanceId)` method should call the relevant objects and methods to load the following instance files according to the mapping: `{ 0: square.obr, 1: libraries-15.obr, 2: carparks-40.obr, 3: tramstops-85.obr, 4: grid.obr, 5: clustered-pois.obr, 6: chatgpt-instance-100-PoIs.obr }`. You may add your own instances from `instanceId` 7+.

Instance Reader

You should complete the implementation of the `OBRInstanceReader` class such that the call to *`readOBRInstance(…)`* is able to read in an OBR problem instance from file and return the problem information as an instance of a `OBRInstanceInterface`. You may use `OBRInstanceReader.java` as a starting point.

The format of an OBR problem instance is as follows, noting that underscores are used here to denote whitespaces or any kind.

1. A single line starting “NAME_:_” and followed by the name of the problem instance.
2. A single line starting "COMMENT_:_” and followed by a comment.
3. A single line reading “BUS_DEPOT_LOCATION” (actual underscores) to denote the following line contains coordinates of the bus depot.
4. Immediately following the previous line are two integers separated by a whitespace. The first integer is the x-coordinate and the second integer the y-coordinate of the location of the bus depot.
5. A single line reading “POINTS_OF_INTEREST” (actual underscores) to denote that the remaining lines in the file up to and excluding the line “EOF” are the locations of each point of interest.

6. A list of locations encoded in the same way as (4.) where each new line is a new and unique point of interest.
7. A terminating line "EOF" to denote the end of the file.

An example of an OBR instance is as follows:

```
NAME : <instance name>
COMMENT : <some comment(s)>
BUS_DEPOT_LOCATION
0 0
POINTS_OF_INTEREST
0 5
5 0
10 0
10 5
10 10
5 10
0 10
EOF
```

Solution

You should complete the implementation of the `OBRSolution` and `SolutionRepresentation` classes to perform the necessary functions and ensuring adherence to the HyFlex API. Here you need to be careful with the difference between deep and shallow cloning of Objects in Java.

Instance

You should complete the implementation of the `OBRInstance` class to provide the necessary problem instance information to the various classes where you may reference this. The `createSolution(InitialisationMode mode)` method should be implemented to generate and return a complete and feasible solution based on one of two methods determined by the `InitialisationMode`.

RANDOM: In this mode the initial solution should be generated randomly based on the seeded random number generator.

CONSTRUCTIVE: In this mode, you should implement the nearest neighbour greedy algorithm where subsequent points-of-interest are chosen as the one that is closest to the current location. If there are multiple locations with equally "best" distances, the next PoI should be visited pseudo-randomly from the subset of best locations.

Low-level Heuristics

At a minimum, you should implement all the heuristics defined below. **Better projects** will consider efficient implementations of the heuristics (think about concepts discussed in the lectures), and implementing additional heuristics beyond what is described below will also attract more marks (consider the problem and representation and what are good heuristics for this type of

problem). **For higher marks**, you should think about other neighbourhood operators which can be used to create additional heuristics and aim to **implement at least**: one other mutation type heuristic, one other local search type heuristic, and one other crossover type heuristic. **Appropriate choices** of heuristics will also allow your selection hyper-heuristic to select those better heuristics so will likely benefit from good choices here and this should reflect in the mark awarded for your solution to the unseen instance during the in-lab practical exam.

AdjacentSwap: This mutation type heuristic swaps adjacent elements of the representation of the solution. It should be possible that the very first and last PoIs can also be swapped if the last PoI in the representation is chosen. This heuristic should make use of the *intensityOfMutation* setting to perform an internal number of swaps equal to:

Intensity of mutation range	Number of internal swaps
$iom \in [0.0, 0.2)$	1
$iom \in [0.2, 0.4)$	2
$iom \in [0.4, 0.6)$	4
$iom \in [0.6, 0.8)$	8
$iom \in [0.8, 1.0)$	16
$iom == 1$	32

Reinsertion: This mutation type heuristic removes a single element and reinserts it into a different position of the solution. It should be possible to choose any element for removal, and it should be possible that it gets reinserted at any **other** position. This heuristic should make use of the *intensityOfMutation* setting to perform an internal number of reinsertions equal to:

Intensity of mutation range	Number of internal reinsertions
$iom \in [0.0, 0.2)$	1
$iom \in [0.2, 0.4)$	2
$iom \in [0.4, 0.6)$	3
$iom \in [0.6, 0.8)$	4
$iom \in [0.8,1.0]$	5

NextDescent: This local search (hill climbing) type heuristic uses the adjacent swap neighbourhood operator to explore the neighbouring solutions. The first improvement that it finds should be persisted and the heuristic returns that as the new solution. Upon each iteration, the heuristic should start scanning from a randomised position within the representation as opposed to only starting from the first position, looping around to attempt all swaps if needed. This heuristic should make use of the *depthOfSearch* setting to perform an internal number of iterations equal to:

Depth of search range	Number of internal iterations
$dos \in [0.0, 0.2)$	1
$dos \in [0.2, 0.4)$	2

$dos \in [0.4, 0.6)$	3
$dos \in [0.6, 0.8)$	4
$dos \in [0.8, 1.0]$	5

DavissHillClimbing: This local search (hill climbing) type heuristic should operate in the same way as Davis's Bit Hill Climbing (DBHC) as covered in lab 2, but where the bitflip for MAX-SAT is replaced with the adjacent swap neighbourhood operator. This heuristic should make use of the *depthOfSearch* setting to perform an internal number of iterations equal to:

Depth of search range	Number of internal iterations
$dos \in [0.0, 0.2)$	1
$dos \in [0.2, 0.4)$	2
$dos \in [0.4, 0.6)$	3
$dos \in [0.6, 0.8)$	4
$dos \in [0.8, 1.0]$	5

Partially Mapped Crossover (PMX): This crossover type heuristic should take two solutions and create a single solution (chosen randomly) according to the explanation of PMX from the lectures (see lecture 6). There is no requirement to use either of the intensity of mutation or depth of search settings for this implementation.

Additional Note(s)

Important: When implementing the domain and its components, you should think about using relevant techniques discussed in lectures to improve the efficiency of your problem domain implementation. You may be asked during the in-lab exam to explain what this (these) technique(s) is (are), or about a specific technique, and explain how such technique(s) work to improve the efficiency of your implementation.

A2.2 – Learning-based Selection Hyper-heuristic

You must implement a learning-based selection hyper-heuristic different from that in the lab exercises and should not be a copy of an existing hyper-heuristic that has been published or made publicly available in any code repositories. You should be able to explain the components you have used and how these together are a selection hyper-heuristic as opposed to any of the other techniques we have seen in the module. **We will ask you to run this with RANDOM initialisation during the practical examination with a computational budget of 5-nominal minutes – see lab 8 on benchmarking and nominal time -** so it is essential that you have the supporting framework in place to run your hyper-heuristic and design your method accordingly.